

CO4 ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Q1. Smartphone Performance Optimization

Problem: A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

1.1 Understanding the Problem

Modern smartphones run multiple applications simultaneously through multitasking and background process management. Lag while switching between apps typically occurs because the processor's memory subsystem cannot supply instructions and data fast enough when the active working set changes. Each time the user switches context, the processor must fetch a fresh set of instructions and data pages, and if these are not present in the fast on-chip caches, the system suffers cache misses and page faults, both of which introduce significant latency.

1.2 Role of L1, L2, L3 Cache and DRAM

- **L1 Cache:** Located directly inside each CPU core, split into separate instruction (L1-I) and data (L1-D) caches, typically 32–64 KB each. It offers the lowest latency (1–2 CPU cycles) but very limited capacity, so only the most recently used instructions/data of the active app reside here.
- **L2 Cache:** A larger (256 KB – 1 MB), slightly slower (10–15 cycles) private or per-cluster cache that acts as a buffer between L1 and L3, holding data likely to be reused soon, such as recently backgrounded app data.
- **L3 Cache:** A shared, larger cache (2–8 MB) across all CPU cores in the SoC, with latency around 30–40 cycles. It reduces traffic to DRAM by retaining data shared across cores or across recently used apps.
- **DRAM (Main Memory):** Off-chip memory (LPDDR4X/LPDDR5) with much larger capacity (6–16 GB) but latency of 100+ cycles. When an app is switched, if its working set has been evicted from L1–L3, the processor must fetch pages from DRAM, and if DRAM itself is under pressure, from flash storage — causing the perceptible lag.

1.3 Root Cause of the Lag

App-switching lag arises from a combination of: (a) cache pollution, where a newly foregrounded app evicts the cache lines of the app that was just backgrounded; (b) insufficient L2/L3 capacity to hold multiple apps' working sets concurrently; and (c) DRAM bandwidth contention when

several background services and the newly activated app compete for main-memory access simultaneously.

1.4 Proposed Hierarchical Memory Redesign

- Increase L2 cache size per core cluster (e.g., from 512 KB to 1 MB) to retain a larger working set per active thread, reducing the frequency of L3/DRAM accesses during context switches.
- Adopt a victim-cache or exclusive cache policy between L2 and L3 so that evicted L2 lines are preserved in L3 rather than discarded, allowing quick recovery when a backgrounded app is resumed.
- Use way-partitioning or cache QoS (Quality of Service) in the shared L3 cache to reserve a minimum number of ways for the foreground app, preventing background services from fully polluting the cache.
- Employ LPDDR5/LPDDR5X DRAM with higher bandwidth (up to 6400 Mbps) and prefetching logic that predicts the next likely app (based on usage pattern) and pre-loads its critical pages into L3 before the user actually switches.
- Introduce a small, low-latency SRAM-based scratchpad or 'shadow cache' for the last 2–3 recently used apps' critical control structures, effectively creating an additional hierarchy level between L3 and DRAM.

1.5 Expected Outcome

This redesign reduces average memory access time (AMAT) by increasing the effective hit rate across the cache hierarchy, minimizes DRAM traffic during context switches, and improves perceived responsiveness and throughput without significantly increasing power consumption or silicon area.

Q2. Cloud Server Memory Bottleneck

Problem: A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

2.1 Understanding the Problem

Database workloads on cloud servers involve large datasets — often far exceeding physical DRAM capacity — combined with high concurrency from multiple simultaneous queries. High query latency generally results from two compounding factors: (1) inefficient use of the processor cache hierarchy leading to repeated main-memory accesses, and (2) virtual memory paging overhead when the working set exceeds available physical RAM, forcing costly disk (or SSD) I/O.

2.2 Cache Hierarchy vs Virtual Memory Paging

Aspect	Cache Hierarchy (L1/L2/L3)	Virtual Memory Paging
Purpose	Bridges CPU speed and DRAM speed gap	Bridges DRAM capacity and disk/SSD capacity gap
Latency	1–40 CPU cycles	Hundreds of cycles (RAM) to milliseconds (disk page fault)
Managed by	Hardware (transparent to OS)	Operating system + MMU (TLB, page tables)
Granularity	Cache lines (64 bytes)	Pages (typically 4 KB or huge pages 2 MB/1 GB)
Failure mode	Cache miss (fast recovery)	Page fault / thrash (very slow recovery)

For a database server, cache hierarchy issues cause moderate slowdowns (extra DRAM trips), whereas paging issues (especially swapping to disk) cause severe latency spikes and are the dominant contributor to unpredictable query response times under memory pressure.

2.3 Recommended Improvements

- Increase last-level cache (L3) size and adopt NUMA-aware scheduling so that query threads access memory local to their socket, reducing cross-node latency.
- Use huge pages (2 MB/1 GB) instead of standard 4 KB pages to drastically reduce TLB misses and page-table walk overhead for large database buffer pools.
- Increase physical DRAM capacity using high-density, high-bandwidth DRAM (DDR5/HBM for compute-adjacent memory) so that the active dataset fits mostly in RAM, minimizing reliance on paging.
- Replace traditional swap-to-HDD with NVMe NAND Flash-based swap or use it as a caching tier (e.g., Intel Optane-style persistent memory) to reduce the penalty of an unavoidable page fault by an order of magnitude.
- Introduce an intermediate tier of persistent/storage-class memory (e.g., MRAM or 3D XPoint-like technology) between DRAM and SSD to provide a graceful latency gradient rather than a steep cliff from nanoseconds to milliseconds.
- Tune the database buffer pool and OS page-replacement policy (e.g., LRU-K or clock-based algorithms) to keep frequently accessed index pages resident in DRAM.

2.4 Expected Outcome

Combining a tuned cache hierarchy with reduced paging overhead (through huge pages and a faster secondary tier) lowers both average and tail latency of database queries, improving throughput and consistency of response times under heavy concurrent load.

Q3. Autonomous Vehicle Data Processing

Problem: An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

3.1 Understanding the Problem

Autonomous vehicles generate continuous, high-volume sensor data from cameras, LiDAR, radar, and ultrasonic sensors that must be fused and processed within strict real-time deadlines (often single-digit milliseconds) to make safe driving decisions. The memory system must therefore provide extremely high bandwidth for streaming sensor data while also guaranteeing low, predictable (deterministic) latency for safety-critical control loops — and must be robust against power loss, since safety-critical state should not be lost.

3.2 Evaluation of SRAM, DRAM, and MRAM

Technology	Latency	Bandwidth/Density	Volatility	Best Use in AV System
SRAM	Sub-nanosecond, fastest	Low density, high per-cell cost	Volatile	On-chip caches and buffers for real-time sensor-fusion and control-loop computation
DRAM	Tens of nanoseconds	High density, high bandwidth (LPDDR5/GDDR6)	Volatile	Main working memory for perception, mapping, and sensor buffer staging
MRAM	Nanosecond-scale, near-SRAM	Moderate density, non-volatile	Non-volatile	Storing critical state (e.g., last-known-safe trajectory, calibration data) that must survive power loss

3.3 Proposed Memory Hierarchy

- Level 1 (SRAM, on-chip): Tightly-coupled SRAM buffers within the sensor-fusion SoC/ASIC to hold the immediate frame of camera/LiDAR data being processed, providing near-instant access for time-critical perception algorithms.
- Level 2 (High-Bandwidth DRAM): GDDR6 or HBM2e DRAM as main working memory, providing the multi-hundred-GB/s bandwidth needed to stream and buffer raw sensor data (e.g., point clouds and video frames) between capture and inference stages.
- Level 3 (MRAM, non-volatile buffer): A dedicated MRAM bank to continuously log the vehicle's most recent safety-critical decisions and sensor snapshots, ensuring that in the event of a sudden power failure, the last known safe state is preserved for post-incident analysis and safe shutdown.
- Level 4 (NAND Flash storage): Bulk non-volatile storage for long-term logging, map data, and trained model weights, accessed less frequently and with relaxed latency requirements.

3.4 Expected Outcome

This hierarchy maximizes sustained bandwidth for streaming sensor fusion through DRAM, minimizes latency for time-critical decisions through SRAM, and adds a resilience layer via MRAM so that safety-critical state is never lost even under abrupt power interruption — directly supporting the real-time and safety requirements of autonomous driving.

Q4. AI Inference Edge Device

Problem: An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

4.1 Understanding the Problem

Edge AI devices (such as smart cameras, IoT sensors, and wearables) run neural-network inference under tight power and thermal budgets, often on battery. A key challenge is loading trained model weights quickly upon activation while keeping idle and active power consumption to a minimum, since these devices may need to wake from a low-power sleep state and begin inference within milliseconds.

4.2 Comparison of NAND Flash, DRAM, and RRAM

Technology	Read Latency	Power Behaviour	Volatility	Endurance	Suitability
NAND Flash	Tens of microseconds	Low idle power (non-volatile, no refresh)	Non-volatile	Limited write cycles ($\sim 10^4$ – 10^5)	Storing the full trained model persistently

Technology	Read Latency	Power Behaviour	Volatility	Endurance	Suitability
DRAM	Tens of nanoseconds	Requires continuous refresh power even when idle	Volatile	Effectively unlimited	Runtime buffer for active-layer computation during inference
RRAM	Tens of nanoseconds (near-DRAM)	Very low power, no refresh needed	Non-volatile	Moderate-high, improving with newer generations	In-place fast weight loading and potential in-memory computation

4.3 Proposed Hierarchical Memory Structure

- Persistent Storage Tier (NAND Flash): Stores the complete trained model and firmware. Being non-volatile, it retains data with zero power draw while the device is powered off, making it ideal for long-term storage of the model.
- Non-Volatile Fast-Load Tier (RRAM): Frequently used or first-layer model weights are pre-staged in RRAM. Because RRAM has near-DRAM latency but consumes no refresh power, the device can begin inference almost immediately upon wake-up without waiting on a full DRAM reload from flash, and without the standby power cost of keeping DRAM powered.
- Working Memory Tier (DRAM, used selectively): Only powered on and populated with the currently executing layer's activations and intermediate weights during active inference, then placed into self-refresh or powered down between inference bursts to save energy.
- On-chip SRAM cache: A small SRAM buffer within the neural processing unit (NPU) for the innermost loop of matrix-multiply operations, minimizing energy per operation for the most frequently accessed values.

4.4 Expected Outcome

By using RRAM as a non-volatile, low-power staging layer between NAND Flash and DRAM, the device achieves fast 'instant-on' model loading, avoids the standby power cost of keeping DRAM active, and preserves NAND Flash purely for bulk, infrequent storage — collectively minimizing both wake-up latency and average power consumption.

Q5. High-Performance Gaming Console

Problem: A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

5.1 Understanding the Problem

Frame drops during large scene loading in a gaming console occur when the GPU and CPU cannot retrieve textures, geometry, and world-state data fast enough to render each frame within the target time budget (e.g., 16.6 ms for 60 FPS). This is fundamentally a throughput problem: large open-world scenes require streaming gigabytes of asset data from storage into memory and then into the GPU's working set faster than the frame budget allows, and the L3 cache and DRAM subsystem can become a bottleneck if not properly sized and managed.

5.2 L3 Cache vs DRAM Throughput

- **L3 Cache Throughput:** Shared L3 cache offers very high throughput (often several hundred GB/s effective bandwidth) but limited capacity (tens of MB at most), so it can only hold a small, recently accessed subset of scene data — sufficient for steady-state rendering of an already-loaded scene, but insufficient for the burst of new data required during a scene transition.
- **DRAM Throughput:** GDDR6/GDDR6X memory in consoles provides very high raw bandwidth (500+ GB/s) and large capacity (10–16 GB shared between CPU and GPU), but this bandwidth must be shared across CPU compute, GPU rendering, and asset-streaming DMA transfers simultaneously; during large scene loads, streaming traffic can saturate this shared bandwidth and starve the GPU's rendering pipeline, causing frame drops.
- The core issue during scene loading is not L3 cache capacity but a temporary DRAM bandwidth contention between the streaming subsystem (loading new assets from SSD) and the rendering subsystem (reading textures/geometry for the current frame).

5.3 Proposed Memory Hierarchy Enhancements

- Increase L3 (or GPU L2) cache size and adopt a scan-resistant replacement policy so that large sequential asset-streaming reads do not evict the working set that the GPU needs for the current frame.
- Implement a Direct Storage / hardware decompression pipeline (as in modern consoles) so that assets are DMA-transferred from NVMe SSD straight into DRAM/VRAM in compressed form and decompressed on dedicated hardware, reducing the DRAM bandwidth consumed by the streaming path.
- Use asynchronous, prioritized memory-access scheduling (Quality-of-Service arbitration) so that rendering-critical reads are given priority over background asset-streaming writes during a frame, smoothing bandwidth allocation.

- Adopt higher-bandwidth GDDR6X or future GDDR7 memory with wider bus width to raise the ceiling of total available throughput, giving more headroom for simultaneous streaming and rendering traffic.
- Introduce a dedicated on-package SRAM scratchpad (similar to a large last-level cache) for render-target and frequently reused texture data, reducing repeated DRAM round-trips during scene transitions.
- Pre-fetch and pre-load likely-needed assets during loading screens or low-activity moments using predictive streaming based on player position/trajectory, spreading the bandwidth demand over time instead of concentrating it at the moment of the scene transition.

5.4 Expected Outcome

By separating and prioritizing streaming bandwidth from rendering bandwidth, enlarging the effective cache capacity, and adopting higher-throughput DRAM, the console can sustain the required data delivery rate during large scene loads, eliminating the bandwidth starvation that causes frame drops and delivering a smoother, more consistent frame rate.

— *End of Solutions* —